Assignment 1: Lexical & Semantic Retrieval on EB-NeRD and MIND

CS4.406: Information Retrieval & Extraction

News Recommendation Systems

Due: August 27, 2026 (at Quiz-1)

Type	Individual
Code Submission	GitHub Classroom (link on Moodle)
Report Submission	Moodle
Competitions	MIND (Codabench) RecSys 2024 Challenge (Codabench)
Starter Code (EB-NeRD)	`https://github.com/jppol-ai/ebnerd-benchmark`
MIND Dataset	`https://huggingface.co/datasets/yjw1029/MIND`
EB-NeRD Dataset	`https://recsys.eb.dk/dataset/`

Note: *You must register on Codabench and submit to both leaderboards.*

Introduction

Build a working, measured, reproducible pipeline on both news datasets: rank the candidate articles in an impression by click likelihood, using the user's click history, session context, and article content. This assignment focuses on **lexical and semantic retrieval** — the foundation pipeline.

Important

Mandatory: You **must** register for both competitions on Codabench and submit your predictions to both leaderboards. Topping a leaderboard is not a necessity—though always appreciated—and grading is **never** on leaderboard rank. Register here:

- **MIND Competition:** `https://www.codabench.org/competitions/13967/`
- **RecSys 2024 Challenge:** `https://www.codabench.org/competitions/2469/`

Datasets

EB-NeRD (RecSys 2024 Challenge, Ekstra Bladet) [Kruse et al., 2024]: ∼2.7M users, 600M+ impression logs, 120K+ articles with title/abstract/body and provided article embeddings. Demo/small/large bundles let you dial scale gradually. Danish language.

MIND (Microsoft News Dataset) [Wu et al., 2020]: ∼1M users, 160K+ English news articles with title/abstract/body and entity annotations, 15M+ impression logs. MIND-small for fast iteration.

Both tasks natively exercise all three modelling axes:

- **Lexical** — titles/bodies (BM25, TF-IDF)
- **Semantic** — article/text embeddings (Word2Vec, BERT, XLM-RoBERTa)
- **Behavioural** — click history, recency/decay, session context

Rapid news decay makes temporal splitting and freshness handling instructive. Both leaderboards are live, so you can submit and verify your solutions against them.

What You Will Build

1. **Reproducible data pipeline** — download → clean → temporal train/val/test split (never random for interaction data) → a small feature store; one command rebuilds from raw files.
2. **Lexical candidate generation (BM25)** — build an inverted index, retrieve top-K candidates using BM25 scoring over article titles and abstracts.
3. **Semantic candidate generation (embeddings)** — compute or load article embeddings, build an ANN index, retrieve top-K candidates.
4. **Offline evaluation harness** — the official metrics (AUC, MRR, nDCG@5, nDCG@10, plus beyond-accuracy: diversity, novelty, coverage) with at least one slice (cold-start vs. warm, head vs. tail) and bootstrap confidence intervals.
5. **Design note (≤4 pages)** — what you built, choices and alternatives, observations, and where it breaks at 10×.

Part 0: Datasets & Setup

EB-NeRD — download directly from the S3 bucket:

```
# EB-NeRD demo (small, for quick iteration)
wget https://ebnerd-dataset.s3.eu-west-1.amazonaws.com/ebnerd_demo.zip

# EB-NeRD small (for final training)
wget https://ebnerd-dataset.s3.eu-west-1.amazonaws.com/ebnerd_small.zip

# Pre-trained embeddings (optional)
wget https://ebnerd-dataset.s3.eu-west-1.amazonaws.com/artifacts/Ekstra_Bladet_word2vec.zip
wget https://ebnerd-dataset.s3.eu-west-1.amazonaws.com/artifacts/
    google_bert_base_multilingual_cased.zip
```

MIND — download from HuggingFace:

```
wget https://huggingface.co/datasets/yjw1029/MIND/resolve/main/MINDsmall_train.zip
wget https://huggingface.co/datasets/yjw1029/MIND/resolve/main/MINDsmall_dev.zip
```

Competition Registration: Register on both leaderboards:

- **MIND:** https://www.codabench.org/competitions/13967/

- **RecSys 2024 Challenge:** https://www.codabench.org/competitions/2469/

Compute: Use free-tier GPUs (Google Colab, Kaggle, Lightning AI). MIND-small and EB-NeRD demo run on a single free GPU in a few hours.

Part I: Lexical & Semantic Retrieval

Individual work. Due: August 27, 2026 (at Quiz-1).

In this assignment, you will build the foundation: a reproducible data pipeline, lexical and semantic candidate generation, and an offline evaluation harness. You will work with both MIND and EB-NeRD datasets.

Q1. Reproducible Data Pipeline

Build a data pipeline that goes from raw files to a ready-to-use feature store in one command. The pipeline must:

1. **Download** raw data files for both MIND-small and EB-NeRD demo/small
2. **Clean** and parse into a unified schema (articles, behaviors/impressions, click history)
3. **Temporal split** into train/val/test — *never* random split for interaction data. Use time-based splitting (e.g., last N days as test, preceding M days as validation)
4. **Feature store** — a small, reusable store for article features (title, abstract, body, category, entities, embeddings) and user features (click history, recency)
5. **One-command rebuild** — a single script (e.g., `make data` or `python build_pipeline.py`) that rebuilds everything from raw files

Q2. Lexical Candidate Generation (BM25)

Implement BM25 retrieval over article titles and abstracts for both datasets:

1. Build an inverted index over article text (title + abstract)
2. Given a user's click history, construct a query (e.g., concatenate titles of recently clicked articles)
3. Retrieve top-K candidate articles using BM25 scoring
4. Report recall@K (how many ground-truth clicked articles appear in the top-K candidates) for $K \in \{50, 100, 200\}$

Q3. Semantic Candidate Generation (Embeddings)

Implement embedding-based retrieval using the provided article embeddings (or compute your own using BERT/XLM-RoBERTa):

1. Compute or load article embeddings for both datasets
2. Build an ANN index (e.g., FAISS, ScaNN, or brute-force for small scale)
3. Given a user's click history, compute a user representation (e.g., mean-pooled embeddings of clicked articles) and retrieve top-K candidates

4. Report recall@K for $K \in \{50, 100, 200\}$
5. Compare lexical vs. semantic retrieval: which works better? On which slices?

Q4. Offline Evaluation Harness

Implement the official evaluation metrics and slicing:

1. **Metrics:** AUC, MRR, nDCG@5, nDCG@10
2. **Beyond-accuracy:** Intra-list diversity, novelty, coverage
3. **Slicing:** At least one slice — cold-start users (few clicks) vs. warm users, or head articles (popular) vs. tail articles
4. **Confidence intervals:** Bootstrap 95% CI for each metric
5. Run your evaluation harness on both BM25 and embedding-based retrieval results

Q5. Codabench Submission

Generate prediction files and submit to both leaderboards:

- Submit MIND predictions to `https://www.codabench.org/competitions/13967/`
- Submit EB-NeRD predictions to `https://www.codabench.org/competitions/2469/`
- Include screenshots of your leaderboard scores in your design note

Q6. Design Note (≤4 pages)

Write a concise design note covering:

- What you built and key design choices
- Alternatives considered and why you chose what you did
- Observations from experiments (lexical vs. semantic, dataset differences)
- Where your pipeline breaks at 10× scale

Part II: Deliverables & Policies

Q7. Deliverables

1. **Code** (GitHub Classroom): reproducible pipeline, model code, evaluation harness, prediction files, `README.md` with one-command reproduce. No large files — use `.gitignore`.
2. **Design note** (≤4 pages, Moodle): what you built, choices, observations, where it breaks at 10×.
3. **Leaderboard screenshots** from both Codabench competitions.

4. **AI usage log**: all prompts, chat history exports, marking of AI-generated vs. human-written code.

Q8. Git Commit Policy

- Commit frequently with meaningful messages.
- No large files in Git — ignore `*.zip`, `*.pt`, `*.ckpt`, `__pycache__/`, `data/`.
- No force-pushes after the deadline.

Q9. Anti-Gaming

- Report metrics with and without features unavailable at serving time.
- Enforce the behaviour-window boundary — no future-click leakage. Include a test asserting this.

Important

Grading note: Grading is **never** on leaderboard rank. Your grade is based on pipeline correctness, system design, ablation rigour, scale analysis, and design-note clarity.

References

Johannes Kruse, Kasper Lindskow, Saikishore Kalloori, Marco Polignano, Claudio Pomo, Abhishek Srivastava, Anshuk Uppal, Michael Riis Andersen, and Jes Frellsen. Eb-nerd: A large-scale dataset for news recommendation. In *Proceedings of the ACM RecSys Challenge 2024*, 2024.

Fangzhao Wu, Ying Qiao, Jiun-Hung Chen, Chuhan Wu, Tao Qi, Jianxun Lian, Danyang Liu, Xing Xie, Jianfeng Gao, Winnie Wu, and Ming Zhou. Mind: A large-scale dataset for news recommendation. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pages 3597–3606, 2020.